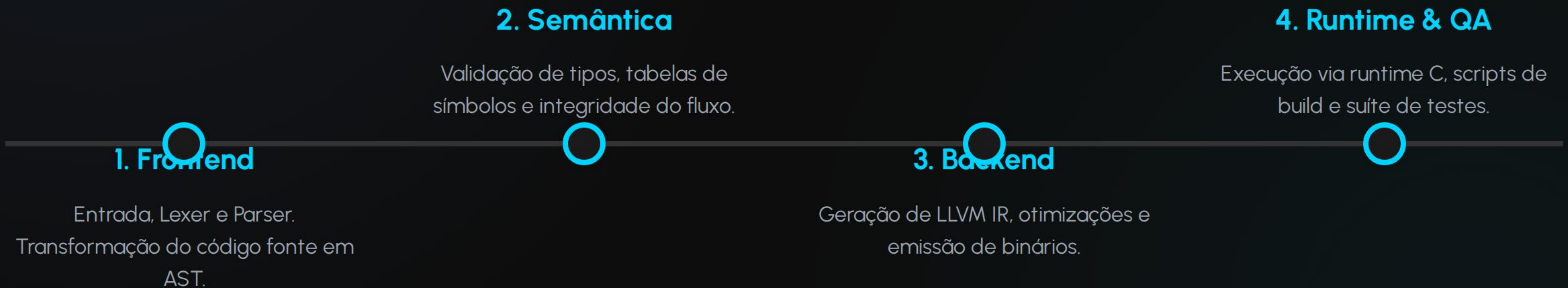


Fluxos de Execução do **mycc-pt**

Uma visão detalhada da arquitetura, pipeline e capacidades do compilador.

O Pipeline de Compilação



Etapa 1: Entrada e Parsing



CLI (Interface)

O binário `mycc_cli` processa argumentos e modos de operação como `--check`, `--dump-ast`, e define flags de otimização.



Lexer

Transforma o texto `.my` em tokens, preservando metadados de linha/coluna e reconhecendo palavras-chave em Português.



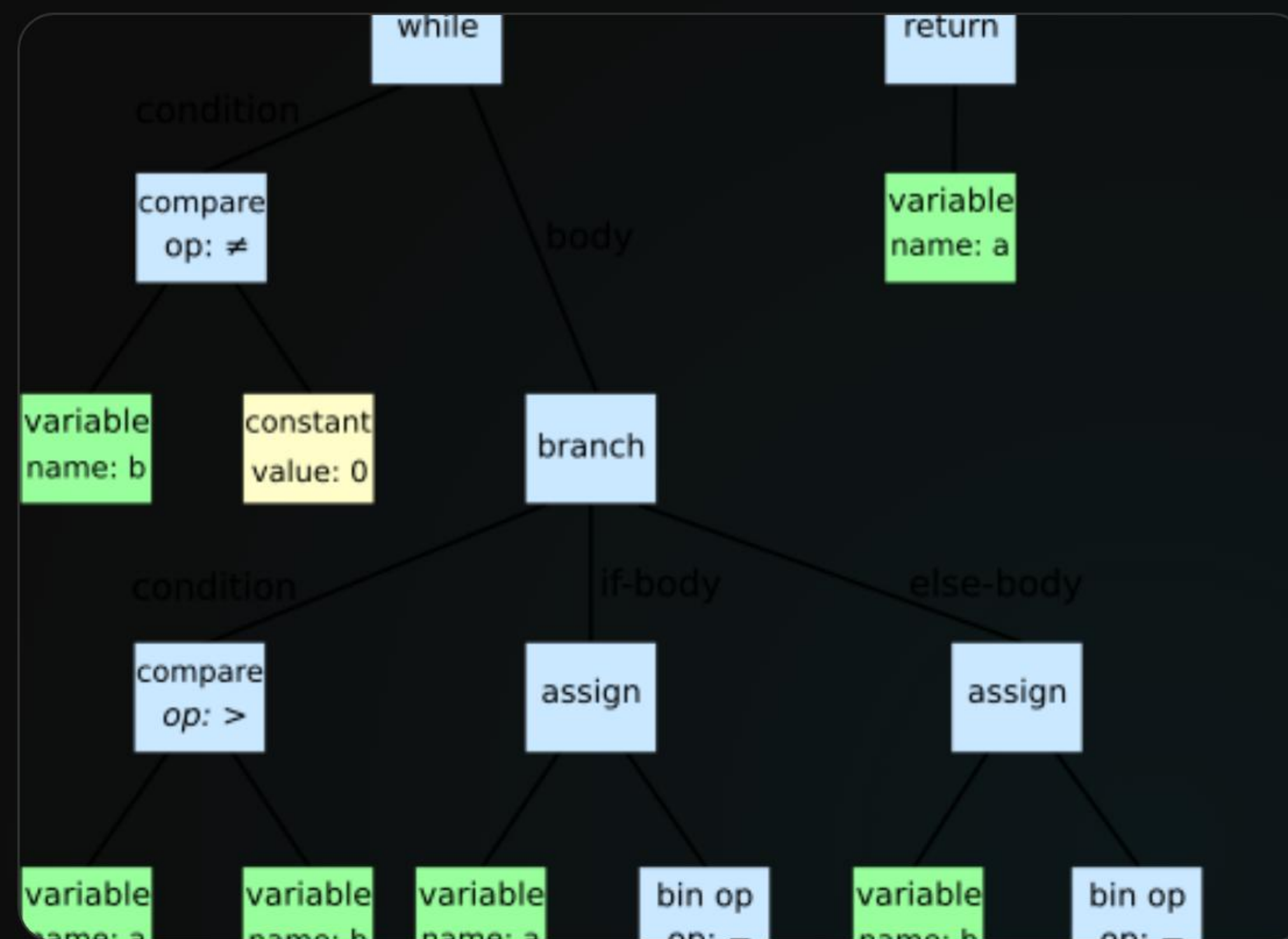
Parser

Consome tokens via descendente recursivo e constrói a **AST** (Árvore Sintática Abstrata) com nós de controle e funções.

Etapa 2: **Análise Semântica**

O módulo `src/semantics.hpp` garante a integridade do código:

- ✓ **Tabelas de Símbolos:** Escopos empilhados para funções e variáveis.
- ✓ **Controle de Fluxo:** Garante que funções retornem valores e que quebra/continua estejam apenas em loops.
- ✓ **Tipos e Memória:** Restringe conversões implícitas e valida limites de vetores.
- ✓ **Diagnósticos:** Emite erros detalhados com localização exata.



Etapa 3: Codegen & LLVM

A AST validada é traduzida para **LLVM IR SSA**, permitindo otimizações avançadas.

- ✓ **Tradução:** Geração de `llvm::Module` com terminadores consistentes.
- ✓ **Otimizações:** Suporte a flags como `--fast2d`, `--vec2d` e `--unroll2d`.
- ✓ **Emissão:** Gera `.ll`, `.bc`, `.asm`, `.obj`, ou executáveis finais.
- ✓ **Execução:** Modo `--run` executa o IR diretamente via JIT.

```
define dso_local i32 @loop_add(i32*, i32) #0 {  
    %3 = alloca i32*, align 8  
    %4 = alloca i32, align 4  
    %5 = alloca i32, align 4  
    %6 = alloca i32, align 4  
    store i32* %0, i32** %3, align 8  
    store i32 %1, i32* %4, align 4  
    store i32 0, i32* %5, align 4  
    store i32 0, i32* %6, align 4  
    br label %7
```

```
label>:7:  
    %8 = load i32, i32* %6, align 4  
    %9 = load i32, i32* %4, align 4  
    %10 = icmp slt i32 %8, %9  
    br i1 %10, label %11, label %22
```

```
label>:11:  
    %12 = load i32*, i32** %3, align 8  
    %13 = load i32, i32* %6, align 4  
    %14 = sext i32 %13 to i64  
    %15 = getelementptr inbounds i32, i32* %12, i64 %14  
    %16 = load i32, i32* %15, align 4  
    %17 = load i32, i32* %5, align 4  
    %18 = add nsw i32 %17, %16  
    store i32 %18, i32* %5, align 4  
    br label %19
```

```
label>:19:  
    %20 = load i32, i32* %6, align 4  
    %21 = add nsw i32 %20, 1  
    store i32 %21, i32* %6, align 4  
    br label %7
```

```
label>:22:  
    %23 = load i32, i32* %5, align 4  
    ret i32 %23
```

; preds = %19, %2

; preds = %7

; preds = %11

; preds = %7



%11:

%12 = load i
%13 = load i
%14 = sext i
%15 = getele
%16 = load
%17 = load i
%18 = add ns
store i32 %18
br label %19

Etapa 3.1: Runtime e Linkagem

Runtime em C

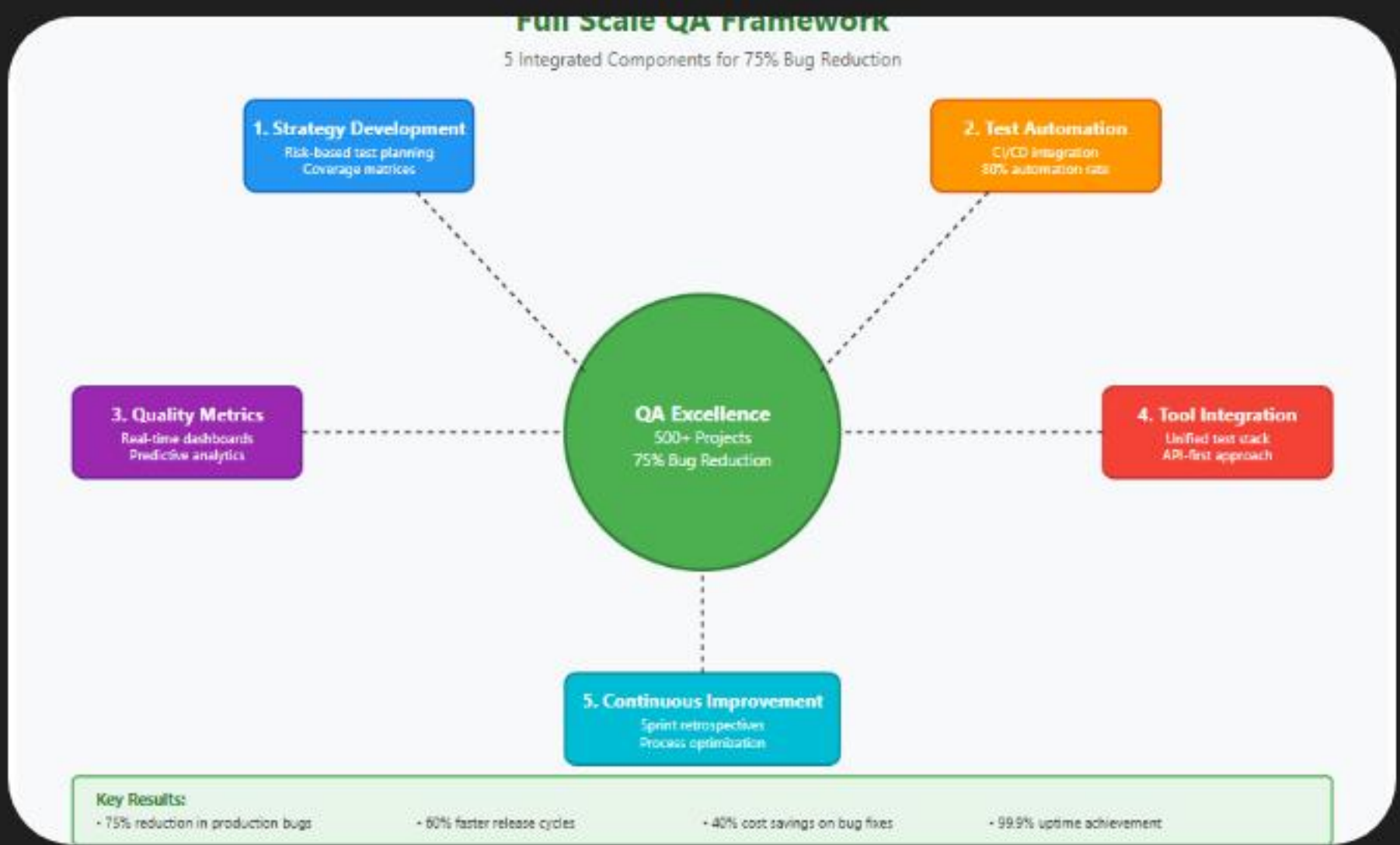
O arquivo `src/runtime.c` fornece a infraestrutura básica necessária para a execução dos programas compilados. Inclui implementações nativas para operações de entrada e saída, como `printf` (inteiros), `printfb` (booleanos) e `prints` (strings).

Processo de Linkagem

Ao utilizar a flag `--emit-exe`, o compilador invoca o `clang`. Ele linka o objeto gerado com a biblioteca estática `libmycc_runtime.a` (usando um caminho embutido), criando um binário final autônomo e pronto para execução.

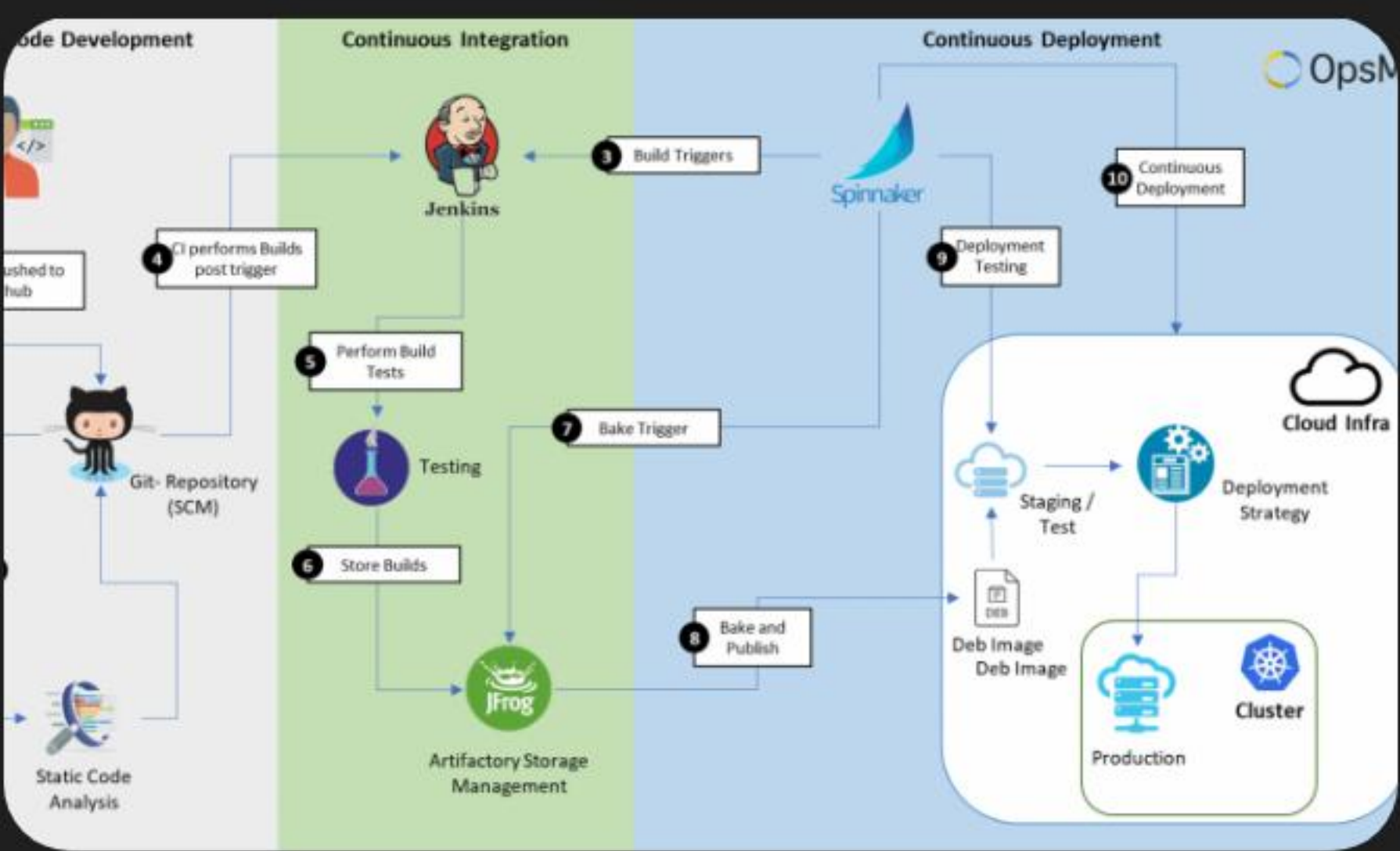
Etapa 4: Scripts e Testes

Garantia de qualidade robusta com mais de 150 casos de teste automatizados.



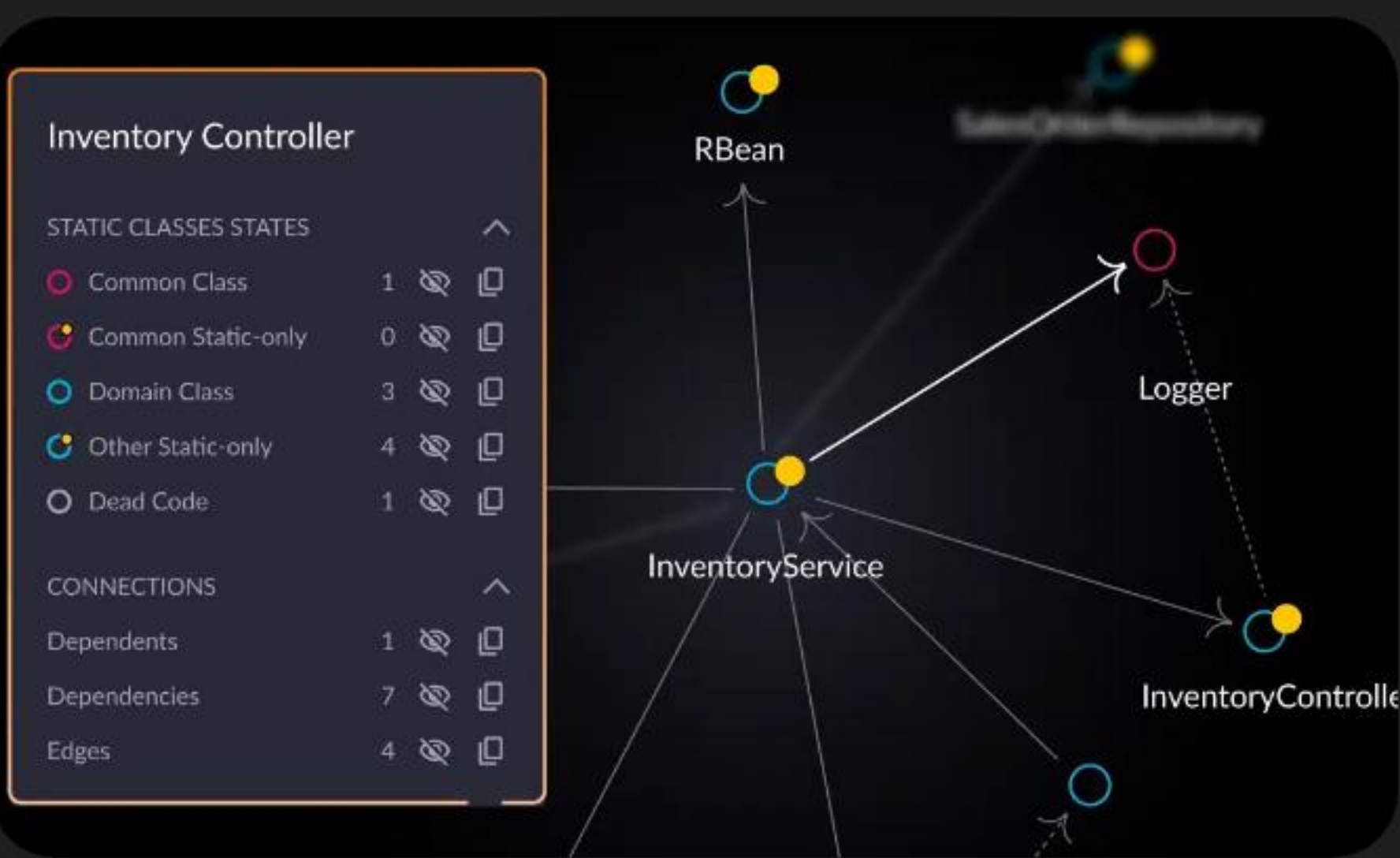
Build Rápido & Clean

CMake + Ninja



Must Pass

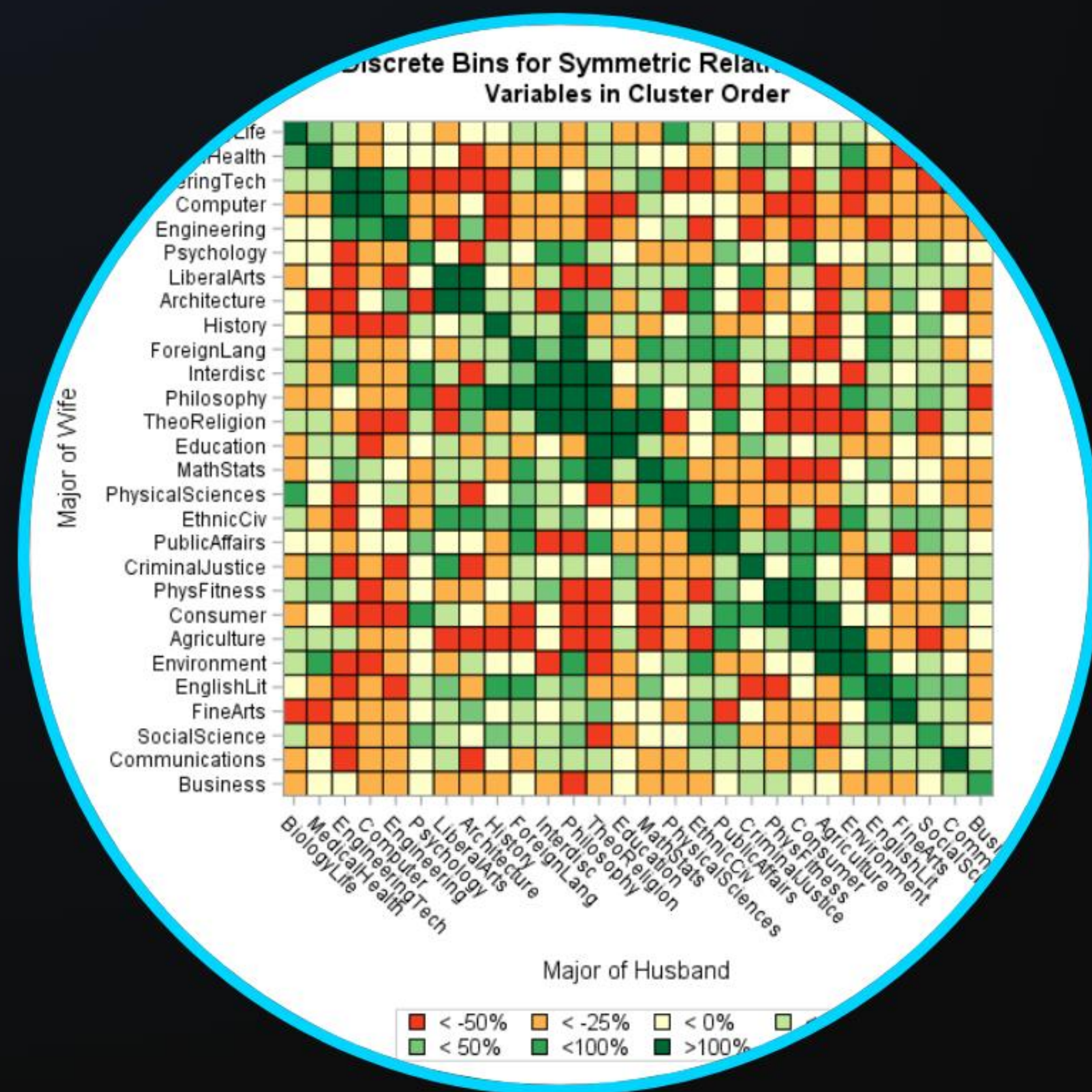
Funcionalidades Core



Must Fail

Erros Semânticos

Possibilidades de Programação



O que você pode criar?

- ✓ **Algoritmos Complexos:** Suporte completo a funções recursivas e loops (busca, ordenação).
- ✓ **Manipulação de Dados:** Vetores 1D e intrínsecas `copy2d/fill2d` para simulação de matrizes.
- ✓ **Controle de Fluxo:** Estruturas robustas com `se/senao`, `enquanto` e `para`.
- ✓ **Demos Interativas:** IO básico para validação lógica e experimentação.

Comandos de Demonstração

Explore as capacidades do compilador através da linha de comando. Desde a visualização da AST até a execução de scripts de teste completos.

Estes comandos permitem verificar a integridade da compilação e visualizar as etapas intermediárias.

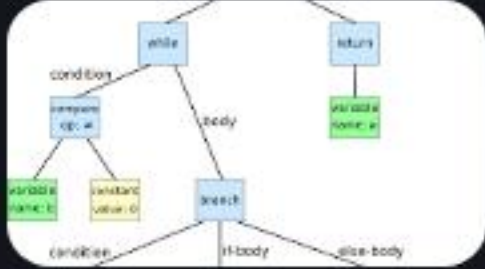
```
$ ./build/mycc_cli --version
$ ./build/mycc_cli --dump-ast exemplo.my
# Gera LLVM IR com verificação
$ ./build/mycc_cli --emit-ll --verify-ir -o out.ll
  exemplo.my

# Executa suite de testes completa
$ ./scripts/run_tests.sh
$ ./scripts/quickstart.sh
```


Perguntas?

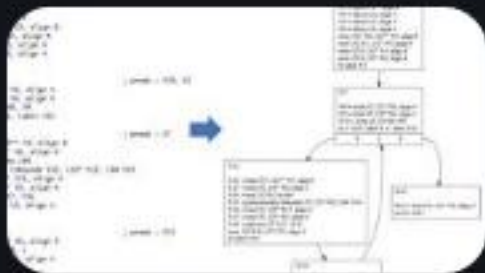
Obrigado pela atenção.

Image Sources



https://upload.wikimedia.org/wikipedia/commons/thumb/c/c7/Abstract_syntax_tree_for_Euclidean_algorithm.svg/500px-Abstract_syntax_tree_for_Euclidean_algorithm.svg.png

Source: en.wikipedia.org



<https://yqintl.alicdn.com/0987cb7f27af7cc16ee8e801c8635aea7265c35a.png>

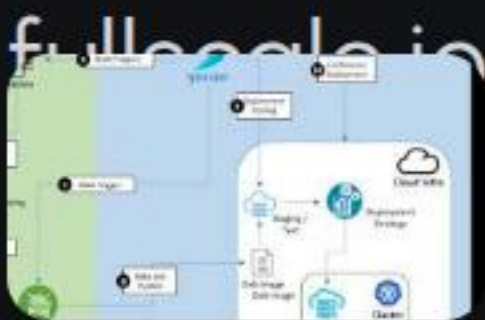
Source: www.alibabacloud.com



Thumbnail
for

https://lh7-rt.googleusercontent.com/docsz/AD_4nXeIKkdnDXAaciJz09BGNJyvwZOn3FNdX7GsP8MzvcCWf7msYLM78zGjRLOkGukFjWi82Flw_Oa50iJM0tSGPUO8SF641bOrPVhdABDScIYsR3-isKuwWrI3ivFNzUpMmdzm6Kr?key=eEulxk-l-ZNxZIUxrnloPQ

Source: fullscale.io



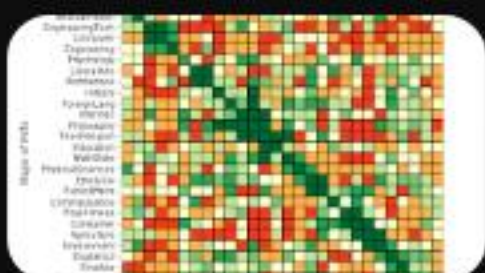
<https://www.opsmx.com/blog/wp-content/uploads/2021/02/CI-CD-Pipeline-e1613664546213.png>

Source: www.opsmx.com



https://vfunction.com/wp-content/uploads/2024/09/blog_static_vs_dynamic_code_analysis_opt_1.webp

Source: vfunction.com



<https://blogs.sas.com/content/iml/files/2018/05/ReorderSingleLinkCluster4.png>

Source: blogs.sas.com